DEPARTMENT OF COMPUTER APPLICATIONS

MASTER OF COMPUTER APPLICATIONS (MCA) DEGREE PROGRAM

MINI PROJECT • FIRST REVIEW DOCUMENT

MULTI-TIER CREDIT CARD FRAUD DETECTION SYSTEM

An End-to-End Banking Framework Combining Supervised Random Forest, Unsupervised Isolation Forest, Real-Time SMS Dispatch, and an Interactive Conversational Support Assistant

Comprehensive Project Report & Architectural Specifications

PROJECT TEAM MEMBERS

1. Rahul R A

MCA Candidate • Machine Learning & Backend Architecture

2. Yedhu Krishnan G

MCA Candidate • Full-Stack Development & API Integration

PROJECT DETAILS

Review Stage: First Review / Design Milestone

Core Technology: Python 3.x, Scikit-Learn, MySQL

Dataset: Benchmark Kaggle Credit Card Fraud Dataset

Integration: Twilio / Fast2SMS API Gateway

1. EXECUTIVE ABSTRACT & METADATA

Academic Abstract: With the rapid global expansion of digital payments, credit card fraud poses severe financial and security threats to banking infrastructures and customer trust. This project introduces a multi-layered, automated **Credit Card Fraud Detection System** integrated with a simulated full-stack core banking application. To simultaneously mitigate known fraud vectors and detect emerging zero-day anomalies, the architecture deploys a **hybrid machine learning pipeline** comprising a supervised **Random Forest classifier** (trained on the benchmark Kaggle transaction dataset) and an unsupervised **Isolation Forest anomaly detector**.

Every card transaction is evaluated in real time to generate a consolidated risk score bounded between 0 and 100. Based on calibrated security thresholds, transactions are dynamically routed: low-risk requests are approved instantly, moderate-risk transactions trigger automated **SMS notifications** to the customer's registered phone number, and high-risk events result in immediate card deactivation. Additionally, an integrated conversational **support chatbot** assists cardholders in querying alert rationales, reviewing recent transactions, and requesting instant card freezes. A centralized **bank admin portal** provides real-time fraud monitoring, manual review capabilities, card management, and comprehensive reporting.

Keywords: Credit Card Fraud Detection, Hybrid Machine Learning, Random Forest, Isolation Forest, Anomaly Detection, Twilio Gateway, Support Chatbot, Relational Database Design.

Project Objectives

- Accurately detect known fraud patterns using supervised classification.
- Identify novel, unseen fraud behaviors via unsupervised anomaly detection.
- Provide instant two-way alerting via an SMS gateway API.
- Empower customers with a 24/7 rule-driven self-service support chatbot.
- Enforce hard security barriers (e.g., automated lockout upon 3 failed PIN attempts).

Technical Stack

Language:	Python 3.10+
Machine Learning:	Scikit-learn (Random Forest, Isolation Forest)
Database:	MySQL 8.0 (Normalized Relational Schema)
Frontend:	HTML5, CSS3, Bootstrap 5.3, JavaScript
API Gateway:	Twilio / Fast2SMS REST API
Dataset:	Kaggle Credit Card Fraud (284,807 txns)

2. DEPARTMENTAL RECOMMENDATIONS & IMPLEMENTATION MATRIX

All departmental guidelines received during preliminary guidance have been rigorously mapped into direct engineering features:

Departmental Guideline	Engineering Implementation	Outcome & Verification
Role-Based Authentication (RBAC)	Dual authentication pipelines for Customer and Bank Admin roles with distinct session tokens.	Prevents horizontal and vertical privilege escalation; verifies role authorization per route.
Secure Data Handling	Passwords hashed with bcrypt/SHA-256; card numbers masked (e.g., `XXXX-XXXX-XXXX-4821`); parameter binding.	Mitigates SQL injection, credential theft, and sensitive cardholder disclosure in UI and logs.
Normalized Database Design	7 relational tables in Third Normal Form (3NF) linked via explicit primary and foreign keys.	Maintains data integrity across transactions, risk alerts, SMS logs, and chatbot sessions.
Hybrid Decision Logic	Supervised Random Forest (0.7 weight) combined with Unsupervised Isolation Forest (0.3 weight).	Combines historical precision with zero-day outlier adaptability.
Proactive Alerting & Audit Trails	Every flagged transaction creates persistent records in `FRAUD_ALERT` and `SMS_LOG`.	Full forensic audit capability and timestamped delivery accountability.

3. MODULAR SYSTEM ARCHITECTURE

The system is organized into seven cohesive modules designed around the principles of separation of concerns and high maintainability:

#	Module Name	Functional Responsibility	Data Stores Used
M1	**Customer Portal**	Account registration, authentication, dashboard overview, masked card viewing, transaction history, and self-service block requests.	`CUSTOMER`, `CARD`
M2	**Admin Control Center**	Bank manager login, real-time transaction inspection, fraud alert review, manual card freeze/unfreeze, and statistical reporting.	`ADMIN`, `CARD`, `FRAUD_ALERT`
M3	**Transaction Engine**	Card validity verification, expiry validation, available balance/credit limit enforcement, and PIN attempt tracking.	`CARD`, `TRANSACTION`
M4	**Hybrid ML Fraud Engine**	Feature scaling, dual-model scoring (Random Forest + Isolation Forest), consolidated risk calculation (0–100), and policy routing.	`TRANSACTION`, `FRAUD_ALERT`
M5	**Alert & Audit Dispatcher**	Generates persistent risk alerts, tracks administrative review decisions, and logs forensic details (hit rules, model scores).	`FRAUD_ALERT`, `TRANSACTION`
M6	**Customer Support Chatbot**	Interactive assistant: answers cardholder questions, explains alert reasons, displays transaction history, and creates block requests.	`CHAT_LOG`, `TRANSACTION`, `CARD`
M7	**SMS Notification Gateway**	Formats templates under 160 characters, connects to Twilio / Fast2SMS REST API, and monitors delivery status lifecycle.	`SMS_LOG`, `CUSTOMER`, `FRAUD_ALERT`

4. HYBRID MACHINE LEARNING METHODOLOGY

1. Supervised Random Forest Classifier

Trained on the highly imbalanced Kaggle dataset (284,807 transactions, 492 frauds, 0.17% fraud rate). Utilizes an ensemble of decision trees to generate a fraud probability score:

```
P_RF = Trees_Predicting_Fraud / Total_Trees
```

Excels at detecting recurring, previously learned fraud patterns with high precision.

2. Unsupervised Isolation Forest

Requires no labels. It recursively partitions features using random splits. Anomalies are isolated near the root with short path lengths:

```
s(x, n) = 2^(-E[h(x)] / c(n))
```

Configuration: `n_estimators=100`, `contamination=0.0017`. Catches zero-day, novel attack patterns.

Unified Dynamic Risk Scoring Formulation

`Risk_Score = [ 0.7 × P_RF × 100 ] + [ 0.3 × Score_IsolationForest × 100 ]`

Weighting Rationale: 70% supervised confidence on established fraud signatures + 30% unsupervised outlier detection.

Dynamic Action Threshold Matrix

Risk Score Band	Decision Action	System Behavior	Notification & Audit
0 – 39 (LOW)	**Auto-Approved**	Transaction executes seamlessly; balance deducted; transaction marked `APPROVED`.	Recorded in standard transaction ledger. No SMS dispatch needed.
40 – 69 (MEDIUM)	**Flagged Alert**	Transaction allowed or put on temporary verification; flagged in `FRAUD_ALERT`.	Instant SMS alert sent to cardholder. Chatbot ready to explain flagging reason.
70 – 100 (HIGH)	**Card Blocked**	Transaction instantly declined; card status transitioned to `BLOCKED` immediately.	Emergency SMS sent to customer and alert flagged on Admin Command Center.
SECURITY OVERRIDE	**3+ Failed PINs**	Card locked instantly by deterministic rule regardless of the ML score.	"Card Locked due to repeated incorrect PINs" SMS dispatched.

5. REAL-TIME SERVICES: SUPPORT CHATBOT & SMS GATEWAY

Conversational Customer Assistant (Chatbot)

Embedded directly in the customer dashboard to provide 24/7 self-service without requiring call center delays.

- **Intent `txn_history`:** Displays last 5 transactions with card numbers masked.
- **Intent `why_flagged`:** Explains model risk score (e.g., "*Risk Score 62: Unusually large amount*").
- **Intent `block_card`:** Requires explicit two-step user confirmation, then updates card status to `BLOCKED`.
- **Intent `report_fraud`:** Flags transaction for priority admin manual review.

Security Guardrail: Works strictly within authenticated sessions. Never returns full card numbers, CVV, or passwords.

SMS Alert Gateway Architecture

Integrated via cloud REST APIs (Twilio / Fast2SMS) using secured environment variables for API keys.

Template Example (<160 chars):
"BANK ALERT: Rs.24,500 spent on card XX4821 looks suspicious (Score: 62). Not you? Open app chatbot or call bank to freeze."

- **Delivery State Machine:** `QUEUED` → `SENT` → `DELIVERED` / `FAILED`.
- **Automatic Retry:** Up to 3 retries on network timeout before marking `FAILED` in `SMS_LOG`.
- **PCI Compliance:** Messages contain masked digits only (e.g., `XX4821`).

6. NORMALIZED RELATIONAL DATABASE ARCHITECTURE (7 TABLES)

The persistence tier is built on MySQL 8.0, partitioned into 7 third-normal-form (3NF) relational tables ensuring transactional ACID consistency:

Table Name	Primary & Foreign Keys	Key Attribute Columns	Functional Role
`CUSTOMER`	`customer_id` (PK)	`full_name`, `email`, `phone`, `password_hash`	Stores user profile and contact credentials for SMS.
`CARD`	`card_id` (PK) `customer_id` (FK)	`card_number`, `credit_limit`, `failed_pin_attempts`, `status`	Maintains card states: `ACTIVE`, `BLOCKED`, `FLAGGED`.
`TRANSACTION`	`txn_id` (PK) `card_id` (FK)	`amount`, `merchant`, `location`, `txn_time`, `status`	Immutable ledger of processed financial attempts.
`FRAUD_ALERT`	`alert_id` (PK) `txn_id` (FK)	`risk_score`, `rf_prob`, `iso_score`, `rule_hit`, `status`	Stores predictive ML outputs and triage decision logs.
`ADMIN`	`admin_id` (PK)	`username`, `password_hash`, `role`, `created_at`	Privileged credentials for bank fraud analysts and officers.
`SMS_LOG`	`sms_id` (PK) `alert_id` (FK)	`phone`, `message`, `sms_status`, `sent_time`	Audit trail for telephony delivery confirmation and retries.
`CHAT_LOG`	`chat_id` (PK) `customer_id` (FK)	`user_message`, `intent`, `bot_reply`, `chat_time`	Maintains conversation transcripts and self-service action logs.

7. ARCHITECTURAL DATA FLOW SPECIFICATIONS

7.1 Level-0 Context Data Flow Diagram (Context DFD)

The Level-0 diagram models the boundary of the Credit Card Fraud Detection System and its communication with external entities (Customer, Bank Administrator, and SMS Gateway API):

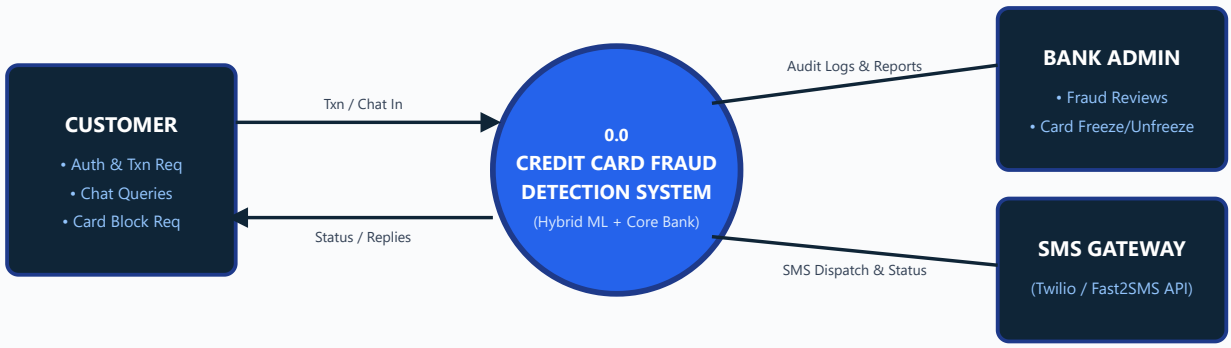


Figure 7.1: Level-0 Context DFD illustrating core interactions across User, System, and External Services.

7.2 Level-1 Functional Process Flow

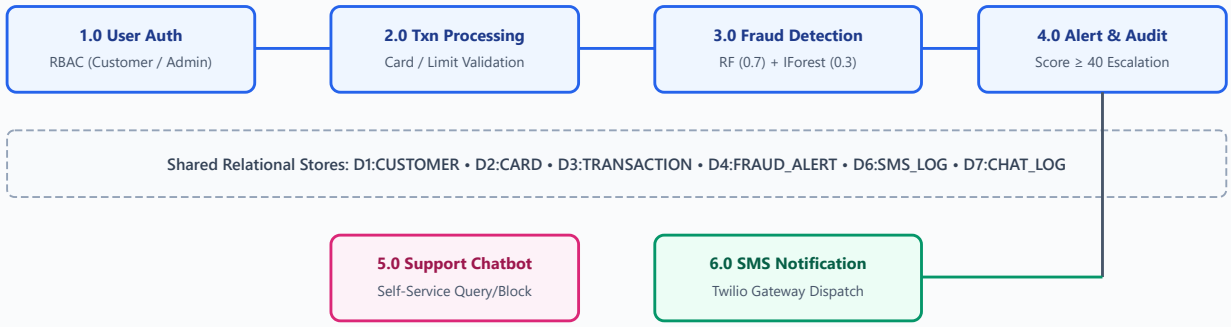


Figure 7.2: Level-1 DFD illustrating execution pipelining between Transaction, ML Scoring, Chatbot, and SMS.

8. COMPREHENSIVE VIVA & EXAMINER DEFENSE GUIDE

Preparation answers for technical queries typically posed by project evaluation committees:

Q1: Why combine Supervised Random Forest with Unsupervised Isolation Forest?

Defense: Supervised models (Random Forest) are exceptionally accurate at recognizing *previously observed fraud patterns* with high recall. However, fraudsters constantly innovate new patterns that supervised models have never seen before. Isolation Forest is an unsupervised outlier detector that isolates rare points near the tree roots without needing labels, making it ideal for catching *novel, zero-day fraud*. The 70/30 dynamic ensemble balances known pattern detection with emerging anomaly adaptability.

Q2: The Kaggle dataset features are PCA components (V1–V28). How does your web app handle this?

Defense: In real commercial banks, $V1$ through $V28$ are transformed behavioral dimensions (e.g., velocity of transactions, geodistance from previous swipe, device fingerprint deviation) anonymized via Principal Component Analysis (PCA) for privacy. For our prototype portal, when a user enters transaction details, our preprocessing pipeline maps behavioral parameters to the PCA vector representation or references a test pool vector to simulate live classification.

Q3: How do you address the severe class imbalance in the credit card dataset?

Defense: In the Kaggle dataset, only 492 out of 284,807 transactions are fraudulent (0.17%). Evaluating purely on accuracy would yield a misleading 99.83% score by simply predicting everything as legitimate. We address this by: (1) using **Precision-Recall AUC** and **F1-score** rather than raw accuracy; (2) tuning `class_weight='balanced'` in Random Forest; and (3) configuring the `contamination=0.0017` parameter in Isolation Forest to match the known base rate.

Q4: Why maintain an independent security rule for 3 failed PIN attempts outside the ML model?

Defense: Machine learning generates probabilistic classifications, but brute-force PIN guessing is a deterministic security breach. Waiting for an ML model score could lead to false-negative delays. Therefore, our Transaction Processing Module enforces a deterministic rule: upon the third consecutive failed PIN attempt (`CARD.failed_pin_attempts ≥ 3`), the card is immediately locked at the database tier regardless of model output.

Q5: What happens if the third-party SMS API fails during a transaction alert?

Defense: The SMS module uses an asynchronous background queue pattern. When an alert triggers, an entry is created in `SMS_LOG` with status `QUEUED`. If the gateway encounters a network timeout or API error, the task automatically retries up to 3 times with exponential backoff before transitioning to `FAILED`. The core transaction decision is not held hostage by SMS gateway latency.

Project Candidates: Rahul R A & Yedhu Krishnan G
Master of Computer Applications (MCA)

Verification Status: Document Approved for First Review
Academic Year 2026 • Mini Project Phase